

Exécution de sprint et qualité logicielle en contexte client réel

Travail de recherche appliquée — 420-D10-AG, Développement avancé d’application mobile

Projet client : application mobile de carte rechargeable — *Eau Renaissance* **Équipe :** [[Équipe X — noms des membres]] **Session :** Hiver 2026 | **Enseignant·e :** [[nom]] | **Date de remise :** [[date]]

Note de configuration technique. Le rapport est rédigé pour une application **Flutter (Dart)** ciblant Android en priorité, avec un backend [[Node.js/Express]] exposant une API REST. Les outils nommés sont ceux de l’écosystème Flutter : `flutter_test` (tests unitaires et de widgets), `mocktail` et `MockClient` du paquet `http` (simulation des appels réseau), `integration_test` (bout en bout sur appareil réel), `flutter_analyze` et `dart format` (qualité statique), `sentry_flutter` (suivi des erreurs), Firebase App Distribution (build installable pour la démo) et FVM (verrouillage de la version du SDK Flutter). Remplacez [[Node.js/Express]] par votre backend réel (Firebase, Supabase, .NET...) ; la logique des cinq axes reste identique.

1. Problématique de l’équipe

Eau Renaissance vend de l’eau traitée en points de distribution. Le mandat confié à l’équipe est une application mobile de **carte rechargeable** : le client crée un compte, associe une carte physique (code QR), recharge un solde, paie un prélèvement d’eau au comptoir et consulte son historique. L’application touche donc à de l’argent et à un solde qui doit rester exact, même lorsque le réseau cellulaire est faible dans les points de distribution.

Durant les deux premiers sprints, l’équipe a livré du code, mais pas de façon fiable. Trois symptômes sont revenus :

- Perte de traçabilité.** Des commits arrivaient directement sur `main` sans issue associée, avec des messages comme « fix » ou « update ». À la revue de sprint, personne ne pouvait dire quelle demande du client correspondait à quel changement, ni pourquoi une fonctionnalité annoncée n’était pas visible.
- Instabilité à la démo.** La fonctionnalité de recharge fonctionnait sur le poste du développeur, mais le solde s’est désynchronisé devant le client lors de la démo du sprint 2 — un cas typique de régression non détectée, faute de tests et de vérification sur un appareil réel.
- Portée mouvante.** Les nouvelles demandes formulées pendant la démo (reçu PDF, remboursement partiel) étaient intégrées immédiatement au travail en cours, sans passer par le backlog, ce qui a fait déborder le sprint et rendu l’engagement de l’équipe non tenable.

Ces symptômes pointent tous vers la même carence : l’équipe possède des compétences techniques, mais **aucun processus explicite et partagé** reliant une demande du client à un artéfact vérifiable. D’où la question de recherche retenue :

Quelles pratiques concrètes permettent à notre équipe de livrer une fonctionnalité mobile stable, démontrable et traçable à chaque sprint, dans le contexte réel du client Eau Renaissance ?

Trois critères opérationnels définissent le succès pour nous. **Stable** : aucune régression bloquante sur le parcours de recharge lors de la démo, vérifiée sur au moins un appareil Android physique. **Démontrable** : chaque élément annoncé « terminé » peut être montré au client depuis un build installable, pas depuis un environnement de développement. **Traçable** : toute ligne de code livrée remonte, en moins de deux clics, à une issue et à un critère d’acceptation approuvé par le client.

2. Méthode de recherche

La démarche suit les six étapes proposées (cerner, rechercher, comparer, décider, appliquer, mesurer) et s’étale sur deux sprints de deux semaines.

Cerner. Nous avons tenu une rétrospective structurée à la fin du sprint 2 et dépouillé nos propres artéfacts : historique Git ([[n]] commits), tableau Jira, notes de la démo client. Chaque irritant a été reformulé en problème mesurable (ex. : « [[x]] % des commits sans référence d’issue »). Ces mesures constituent notre **base de comparaison (baseline)**, consignée à l’annexe E.

Rechercher. Nous avons privilégié trois types de sources : (a) des sources normatives, pour les définitions à ne pas réinventer — le *Scrum Guide* (Schwaber et Sutherland, 2020), la spécification *Conventional Commits* (2023), le MASVS de l’OWASP (2024) ; (b) de la documentation officielle d’outils, pour ce qui est vérifiable directement — GitHub Docs, Atlassian Support, la documentation Flutter (tests, tests d’intégration) et Firebase ; (c) des sources d’ingénierie reconnues pour l’argumentaire de conception — Vocke (2018) sur la pyramide de tests et Humble et Farley (2010) sur la livraison continue. Les billets de blogue non signés et les tutoriels sans date ont été écartés.

Comparer. Pour chacun des cinq axes, nous avons retenu de deux à trois pratiques réellement envisageables dans notre contexte et les avons évaluées selon quatre critères pondérés, choisis pour refléter nos contraintes d’étudiants : *coût d’adoption* (temps d’apprentissage et de mise en place), *effet sur la stabilité de la démo*, *effet sur la traçabilité*, *soutenabilité à 4 personnes* sur un sprint de deux semaines. Nous avons volontairement écarté les pratiques qui exigent un rôle dédié (ingénieur QA, *release manager*) ou une infrastructure payante.

Décider, appliquer, mesurer. Chaque pratique retenue a été transformée en tâche Jira du sprint courant, avec une personne

responsable. L’effet est mesuré en comparant les mêmes indicateurs qu’à la base de comparaison, à la fin du sprint 3.

Limites de la méthode. L’échantillon est d’un seul projet et de quatre personnes ; les écarts observés peuvent être dus à l’apprentissage du domaine plutôt qu’aux pratiques elles-mêmes. Deux sprints ne suffisent pas à mesurer la durabilité d’une pratique. Nous n’avons pas de groupe témoin. Les résultats doivent donc être lus comme une **évaluation interne située**, non comme une généralisation.

3. Analyse comparative des pratiques

3.1 Axe 1 — Workflow d’équipe : issue → branche → PR

Pratique	Avantages	Limites	Conditions de succès
A. Trunk-based sur main , commits directs	Aucune cérémonie ; rapide	Aucune revue ; main cassé bloque toute l’équipe et la démo ; traçabilité nulle	Exige des tests automatisés matures et de petits lots ; irréaliste pour nous
B. Git Flow (develop, release/*, hotfix/*)	Séparation nette des versions	Trop de branches longues pour un sprint de 2 semaines ; conflits de fusion fréquents ; lourd à expliquer au client	Utile avec des versions publiées en parallèle, ce qui n’est pas notre cas
C. GitHub Flow : une branche courte par issue, une PR, fusion après revue	main reste toujours démontrable ; la PR devient le point d’ancrage de la traçabilité ; GitHub relie la PR à l’issue et la ferme automatiquement (GitHub Docs, 2026)	Exige de la discipline de découpage ; risque de PR trop grosses ; dépend de la réactivité des relecteurs	Branches nommées d’après la clé Jira ; PR de moins de ~400 lignes ; délai de revue plafonné

Décision. GitHub Flow, avec une convention de nommage imposée : feat/EAU-42-recharge-carte. Atlassian Support (2026) indique que la présence de la clé de l’issue dans le nom de branche, le message de commit ou le titre de la PR suffit à faire remonter l’activité de développement dans le ticket Jira ; la traçabilité devient donc un **effet secondaire gratuit** d’une convention de nommage, et non un travail de documentation supplémentaire. C’est l’argument décisif dans notre contexte, où personne n’a de temps à consacrer à de la saisie double.

Pour les messages, nous adoptons *Conventional Commits* (feat, fix, test, docs, refactor, chore), suivant la spécification 1.0.0 (2023). Son intérêt réel pour nous n’est pas la génération automatique de journal des versions — nous n’en publions pas — mais le fait qu’un git log --oneline devienne lisible par le client et par l’enseignant, et qu’un fix: signale immédiatement un correctif à vérifier en test de régression. Nous restons critiques : la convention n’apporte rien si le corps du message ne dit pas *pourquoi* ; nous exigeons donc une ligne « Pourquoi » dans les commits de correctif.

3.2 Axe 2 — Qualité de sprint : DoR, DoD, critères d’acceptation

Le *Scrum Guide* (Schwaber et Sutherland, 2020) ne définit qu’un seul de ces artefacts, la **Definition of Done**, présentée comme un engagement attaché à l’incrément : ce qui ne la satisfait pas ne peut être ni relâché ni présenté comme terminé. La *Definition of Ready* n’est pas dans le cadre officiel ; c’est une pratique d’équipe répandue, et cela change la façon de l’utiliser.

Pratique	Avantages	Limites	Conditions de succès
DoD unique et courte	Une seule barre, non négociable ; rend l’incrément démontrable	Peut être trop générique pour des cas mobiles (permissions, hors ligne)	Doit être affichée dans le gabarit d’issue, pas dans un document oublié
DoD par paliers (code / fonctionnalité / version)	Nuance utile en mobile où le déploiement en magasin est lent	Complexité de gestion ; risque de discussions sur le palier applicable	Utile seulement si l’on publie sur les magasins d’applications
DoR formalisée et bloquante	Filtre les demandes floues avant le sprint ; réduit le travail refait	Devient une barrière bureaucratique : des tickets attendent indéfiniment un détail mineur, et l’on retombe dans une logique en cascade	Doit rester une <i>liste de vérification de conversation</i> , révocable par l’équipe, pas un portail d’approbation
Critères d’acceptation en Gherkin (Étant donné / Quand / Alors)	Directement convertibles en cas de test ; langage partagé avec le client	Verbeux ; tentation d’écrire des scénarios d’interface fragiles	Écrire au niveau du comportement métier, pas des composants

Décision. Une DoD unique et courte (annexe A), intégrée au gabarit d’issue GitHub, plus une DoR **non bloquante** : une issue qui n’est pas « prête » n’entre pas au sprint, mais l’équipe peut passer outre par décision explicite consignée en commentaire. Ce compromis est notre principal point de désaccord interne, tranché ainsi parce que notre client est une PME qui répond parfois en 48 h : une DoR strictement bloquante aurait vidé notre sprint.

Les critères d’acceptation sont rédigés en Gherkin pour les parcours critiques seulement (recharge, paiement, solde), en incluant systématiquement deux cas mobiles que nous avions oubliés au sprint 2 : **perte de réseau en cours d’opération** et **refus de la permission caméra**.

3.3 Axe 3 — Stratégie de tests mobile

La pyramide de tests (Vocke, 2018) demeure notre cadre de référence : beaucoup de tests unitaires rapides, moins de tests intermédiaires, très peu de tests de bout en bout. Elle s’aligne presque directement sur la taxonomie officielle de Flutter, qui distingue les tests **unitaires** (une fonction ou une classe, sans rendu), les tests de **widgets** (un composant d’interface rendu dans un environnement de test) et les tests d’**intégration** exécutés sur un appareil ou un émulateur réel (Flutter, 2026a). Cette taxonomie est un atout : le niveau widget, propre à Flutter, permet de tester un écran complet — états de chargement, messages d’erreur, rendu conditionnel — en quelques centaines de millisecondes, alors qu’un test équivalent en natif exigerait un test instrumenté lent (Android

Elle doit néanmoins être adaptée à notre réalité : en mobile, les défaillances qui embarrassent une démo viennent rarement de la logique pure ; elles viennent du cycle de vie de l’application, des permissions, de la latence réseau et de la diversité des appareils. Or ni les tests unitaires ni les tests de widgets n’exercent la caméra, le stockage sécurisé ou une véritable coupure réseau, puisqu’ils tournent dans une machine virtuelle Dart sans matériel. C’est exactement ce qui nous a échappé au sprint 2.

Niveau	Outil retenu	Ce qu’on y met	Ce qu’on n’y met pas	Coût
Unitaire	<code>flutter_test (test())</code>	Calcul de solde en cents (<code>int</code>), validation de montant, idempotence d’une recharge, formatage de devise	Tout ce qui touche à la navigation ou au rendu	Faible
Widget	<code>flutter_test (testWidgets())</code> + <code>mocktail / MockClient</code>	Écran de recharge : saisie → <code>CircularProgressIndicator</code> → succès ou <code>SnackBar</code> d’erreur ; état hors ligne	Appels réseau réels (le client <code>http</code> est injecté et simulé)	Moyen
Intégration / bout en bout	<code>integration_test</code> sur appareil réel (Flutter, 2026b)	Un seul parcours : connexion → scan → recharge de 20 \$ → solde mis à jour → historique	Tous les autres parcours	Élevé
Exploratoire manuel	Grille de 10 min sur appareil physique	Appareil Android bas de gamme, mode avion, rotation, reprise après mise en arrière-plan	—	Faible mais récurrent

Arbitrage critique. Nous avons hésité sur le niveau d’intégration. Trois options s’offraient : se limiter aux tests de widgets (rapides, mais aveugles au matériel), écrire des tests `integration_test` (officiels, exécutables sur appareil et en intégration continue, mais lents et sensibles aux attentes implicites), ou ajouter `patrol` pour piloter les fenêtres système comme les demandes de permission (plus puissant, mais une dépendance supplémentaire à entretenir). Nous retenons `integration_test`, avec un **seul** scénario, celui qui a échoué devant le client. Ce compromis protège précisément le scénario de démo tout en bornant le coût d’entretien ; la contrepartie assumée est que la boîte de dialogue de permission caméra reste vérifiée manuellement (grille de l’annexe C) plutôt qu’automatiquement. Si ce test unique devient intermittent, nous le retirerons plutôt que de le tolérer — un test rouge qu’on ignore est pire que pas de test (Humble et Farley, 2010).

Nous ajoutons deux règles de prévention des régressions : tout `fix` doit être accompagné d’un test qui échouait avant le correctif ; et le parcours de recharge est rejoué manuellement sur appareil physique avant chaque démo, à l’aide de la grille de l’annexe C. Parce que l’application manipule un solde monétaire, nous retenons également du MASVS de l’OWASP (2024) deux vérifications minimales : le jeton de session est conservé dans `flutter_secure_storage` (Keystore Android) et jamais dans `SharedPreferences`, et les montants sont validés côté serveur — la validation dans le widget n’est qu’une commodité d’interface. Le solde affiché est traité comme un cache d’affichage, jamais comme une source de vérité.

3.4 Axe 4 — Debugging, journalisation, escalade

Pratique	Avantages	Limites	Conditions de succès
Débogage local uniquement (console, points d’arrêt)	Immédiat, aucun outil	Ne capte rien de ce qui survient chez le client ou sur un autre appareil ; l’information disparaît à la fermeture	Suffisant pour le développement, insuffisant pour la démo
Journalisation structurée (niveaux, identifiant de corrélation)	Une opération de recharge devient reconstituable de bout en bout ; permet de répondre au client avec des faits	Risque de fuite de données ; bruit si tout est journalisé	Interdire les données sensibles dans les journaux ; identifiant de corrélation partagé application ↔ API
Suivi des erreurs à distance (<code>sentry_flutter</code>)	Capture les plantages réels avec trace de pile et version ; volet gratuit suffisant ; <code>FlutterError.onError</code> et les erreurs asynchrones sont interceptées	Dépendance externe ; les traces d’un build <i>release</i> sont illisibles sans téléversement des symboles de débogage	Téléverser les symboles à chaque build de démo ; associer chaque erreur à la version ; l’intégrer avant la démo, pas après

Décision. Les trois, par couches : journalisation structurée (`logger`, avec niveaux et identifiant de corrélation par transaction, `debugPrint` proscrit hors développement), `sentry_flutter` sur les builds de démonstration, et outils locaux pour le reste — Flutter DevTools et l’inspecteur de widgets pour les problèmes de rendu, points d’arrêt pour la logique. Nous formalisons surtout une **règle d’escalade en trois paliers**, absente jusqu’ici et cause directe de deux blocages du sprint 2 : (1) 45 minutes d’investigation seul, avec notes écrites dans l’issue ; (2) appel à un pair, en binôme, 30 minutes ; (3) au-delà, publication dans le canal d’équipe avec la mention explicite du risque pour la démo, et inscription au registre des blocages (annexe D). La valeur de cette règle n’est pas technique : elle **retire la charge sociale** de demander de l’aide, en la rendant obligatoire au bout d’un délai.

3.5 Axe 5 — Démo client et changement de portée

Pratique	Avantages	Limites	Conditions de succès
Accepter les demandes séance tenante	Client satisfait sur le moment	Détruit l’engagement du sprint ; le travail en cours reste inachevé ; la confiance s’érode au sprint suivant	Aucune ; c’est notre erreur du sprint 2
Refuser jusqu’au sprint suivant	Protège le sprint	Perçu comme rigide par une PME ; peut faire manquer une information critique	Exige une explication claire et une date

Capter, qualifier, replanifier : toute demande devient une issue au <i>backlog</i> durant la démo, sous les yeux du client, puis est arbitrée au raffinement	Le client voit sa demande enregistrée — ce qui suffit le plus souvent ; rend le compromis visible ; conforme à la logique du <i>backlog</i> comme source unique du travail (Schwaber et Sutherland, 2020)	Nécessite qu’une personne tienne le tableau pendant la démo ; nécessite un arbitrage réel, sinon le <i>backlog</i> devient un cimetière	Un rôle d’« interlocuteur client » désigné ; une issue créée en direct avec le nom du demandeur et la valeur attendue
---	---	---	---

Décision. La troisième pratique, avec un déroulement de démo fixe de 20 minutes : rappel des critères d’acceptation annoncés (2 min) ; démonstration sur **appareil physique à partir d’un build installable** (10 min) ; ce qui n’a pas été livré et pourquoi (3 min) ; nouvelles demandes saisies en direct (5 min). Le point le plus important est le build installable : un APK produit par flutter build apk --release et distribué au client par Firebase App Distribution (Firebase, 2026). Un client à qui l’on montre un émulateur sur l’écran du développeur ne peut pas juger de la stabilité, et l’équipe se prive du seul contexte où les problèmes de permissions, de performance en mode *release* et de réseau apparaissent. Précision importante en Flutter : un build *debug* est notablement plus lent qu’un build *release* à cause de la compilation à la volée, si bien qu’une démo faite en mode debug donne au client une impression de lenteur qui n’est pas celle du produit livré.

3.6 Synthèse transversale et esprit critique

Évaluation des pratiques retenues selon nos quatre critères (— défavorable, ○ neutre, + favorable, ++ très favorable) :

Pratique retenue	Coût d’adoption	Stabilité de la démo	Traçabilité	Soutenable à 4
GitHub Flow + convention de nommage	++ (30 min)	+	++	++
Conventional Commits	++	○	+	+
DoD courte dans le gabarit d’issue	++	++	+	++
DoR non bloquante	+	+	○	+
Critères en Gherkin (parcours critiques)	○	++	++	+
Tests unitaires ciblés	○	++	○	++
Un test integration_test	—	++	○	○
Journalisation + sentry_flutter	○	+	+	+
Règle d’escalade 45/30/publication	++	+	+	++
Démo scriptée + build installable	+	++	+	++

Trois constats se dégagent de cette lecture croisée, et ils nuancent le discours habituel sur les « bonnes pratiques ».

Premier constat : la traçabilité et la stabilité ne s’obtiennent pas par les mêmes moyens. Les pratiques les plus fortes en traçabilité (nommage, Gherkin, liaison issue-PR) sont administratives et presque gratuites ; celles qui produisent la stabilité (tests, build installable, vérification sur appareil) coûtent du temps de développement. Une équipe qui n’adopte que les premières obtient un tableau impeccable et une démo qui échoue quand même — ce qui décrit exactement notre sprint 2 après nos premières corrections partielles.

Deuxième constat : le coût réel d’une pratique est un coût d’entretien, pas d’installation. Brancher integration_test prend une demi-journée ; garder ce test fiable coûte à chaque sprint, d’autant qu’un scénario Flutter mal écrit dépend d’attentes implicites (pumpAndSettle) qui se brisent au moindre changement d’animation. C’est pourquoi nous avons plafonné l’engagement à un seul test et prévu explicitement une porte de sortie. À l’inverse, une règle de protection de branche s’active une fois et ne se dégrade pas. À effort disponible constant, nous privilégions donc les pratiques dont le coût est ponctuel plutôt que récurrent.

Troisième constat : plusieurs pratiques ne fonctionnent qu’appariées. Écrire des tests sans intégration continue ne garantit rien, puisque leur exécution repose sur la mémoire de chacun (Humble et Farley, 2010) ; une DoD exigeant la revue par un pair est inopérante sans règle de délai, car la PR attend ; capter les demandes du client sans arbitrage au raffinement transforme le *backlog* en dépôt. Nos recommandations sont donc présentées par paires fonctionnelles dans la section suivante (R4 avec R9, R2 avec la règle des 24 h, R8 avec le raffinement), et non comme une liste d’éléments interchangeables.

Ce que notre comparaison ne peut pas établir. Nos quatre critères sont pondérés par notre situation d’équipe étudiante à effectif fixe et sans rôle QA ; une équipe professionnelle avec un budget de matériel et une chaîne de livraison en place arbitrerait probablement en faveur des tests E2E et d’une matrice d’appareils, que nous écartons ici. Nos jugements de coût reposent de plus sur nos estimations et sur la documentation des outils, non sur une mesure comparée de plusieurs configurations. C’est une limite assumée : l’objectif du travail est de décider dans notre contexte, pas de produire un classement généralisable.

4. Recommandations prioritées

Priorisation par rapport valeur/effort, chaque recommandation étant reliée à une issue du *backlog*.

#	Recommandation	Axe	Effort	Impact attendu	Issue
R1	Protéger main : PR obligatoire, 1 approbation, branche nommée	1	30 min	Traçabilité ; main toujours démontrable	[[EAU-51]]

	type/EAU-xx-sujet				
R2	Gabarit d’issue + de PR contenant DoR, DoD et grille de revue	1-2	1 h	La qualité devient l’option par défaut	[[EAU-52]]
R3	Critères d’acceptation en Gherkin pour le parcours de recharge, incluant hors ligne et refus de permission	2	2 h	Élimine l’ambiguïté avec le client	[[EAU-53]]
R4	Tests unitaires sur le calcul de solde et l’idempotence de la recharge	3	4 h	Prévention des régressions sur la logique monétaire	[[EAU-54]]
R5	Un test <code>integration_test</code> sur le parcours de démo, exécuté sur appareil réel	3	6 h	Protège le scénario présenté au client	[[EAU-55]]
R6	Journalisation structurée (<code>logger</code>) avec identifiant de corrélation + <code>sentry_flutter</code> sur les builds de démo	4	3 h	Diagnostic factuel au lieu de conjectures	[[EAU-56]]
R7	Règle d’escalade 45/30/publication + registre des blocages	4	30 min	Réduit le temps perdu en blocage individuel	[[EAU-57]]
R8	Déroulement de démo fixe + APK <i>release</i> distribué par Firebase App Distribution + saisie des demandes en direct	5	2 h	Contient les changements de portée	[[EAU-58]]
R9	Intégration continue GitHub Actions (<code>subosito/flutter-action</code>) : <code>dart format --set-exit-if-changed</code> , <code>flutter analyze</code> , <code>flutter test</code> sur chaque PR	1-3	3 h	Rend R4 réellement contraignant	[[EAU-59]]
R10	Vérifications MASVS minimales (<code>flutter_secure_storage</code> , validation serveur)	3	4 h	Réduit le risque sur un solde monétaire	[[EAU-60]]

Ce que nous avons écarté, et pourquoi. Couverture de code minimale imposée (mesure facilement contournable, incite à des tests creux avant d’avoir une culture de test) ; matrice d’appareils dans le nuage (coût, et nous avons accès à trois appareils physiques) ; livraison continue vers les magasins d’applications (les délais de revue des magasins n’ont pas de sens à l’échelle d’un sprint de deux semaines).

5. Plan d’implantation

Sprint courant ([[sprint 3, dates]]) — R1, R2, R3, R4, R7. Ces cinq éléments coûtent moins d’une journée-personne au total et transforment le processus dès maintenant. Séquence : R1 et R2 le premier jour ([[responsable]]), R7 au même moment car sans coût technique, puis R3 avec validation du client par courriel avant d’écrire du code, puis R4 pendant le développement de la recharge. **Critère de sortie :** 100 % des PR du sprint sont liées à une issue et revues par un pair ; le parcours de recharge est démontré sur appareil physique.

Sprint suivant ([[sprint 4]]) — R5, R6, R8, R9, puis R10. R9 est placé avant R5 dans l’ordre d’exécution, même s’il est moins prioritaire : sans intégration continue, les tests de R4 ne sont exécutés que par celui qui y pense. **Critère de sortie :** aucune régression bloquante en démo ; au moins trois blocages consignés avec leur action corrective ; toutes les demandes du client saisies comme issues pendant la démo.

Répartition et risques. [[Assigner chaque R à un membre]]. Risque principal : l’instabilité du test `integration_test` (attentes d’animation, lenteur de l’émulateur en intégration continue). Mesure de repli : si le test échoue de façon intermittente deux sprints de suite, on le remplace par la grille manuelle de l’annexe C. Risque secondaire : dérive de la version du SDK Flutter entre les postes, contenue par FVM et par le versionnement de `pubspec.lock`.

6. Mesure de l’effet observé

Indicateur	Avant (sprint 2)	Après (sprint 3)	Cible
Commits reliés à une issue	[[%]]	[[%]]	100 %
PR revues par un pair avant fusion	[[%]]	[[%]]	100 %
Délai moyen de revue de PR	[[h]]	[[h]]	< 24 h
Défauts bloquants durant la démo	[[n]]	[[n]]	0
Éléments annoncés « terminés » mais non démontrables	[[n]]	[[n]]	0
Temps de revue de PR	[[h]]	[[h]]	< 50 %

Temps cumule perdu en blocage	[[h]]	[[h]]	↓ 50 %
Demandes du client capturées comme issues	[[n/n]]	[[n/n]]	100 %

Lecture critique des résultats. [[Rédiger après le sprint 3]] — préciser ce qui s’est amélioré, ce qui a coûté plus cher que prévu, et distinguer l’effet des pratiques de l’effet de l’apprentissage. Deux effets pervers à surveiller : des PR artificiellement découpées pour « faire beau » dans les statistiques, et des approbations de complaisance qui font passer le taux de revue à 100 % sans améliorer la qualité. C’est pourquoi nous suivons *à la fois* un indicateur de processus (taux de revue) et un indicateur de résultat (défauts en démo) : le second seul est difficile à simuler.

7. Conclusion

Aucune des pratiques retenues n’est innovante ; c’est précisément l’intérêt du résultat. Ce qui manquait à l’équipe n’était pas un outil, mais la **fermeture de la boucle** entre une demande du client, une issue, une branche, une revue, un test et une démonstration sur appareil réel. Les pratiques les plus rentables se sont révélées les moins coûteuses — protection de branche, gabarits, convention de nommage — parce qu’elles déplacent la qualité du niveau de la volonté individuelle vers celui du processus par défaut. Les pratiques coûteuses (tests d’intégration, chaîne d’intégration continue) ne valent la peine qu’appliquées au parcours qui fait vivre le produit : pour Eau Renaissance, la recharge du solde. La taxonomie de tests de Flutter nous y a aidés plus qu’on ne l’anticipait : en déplaçant vers le niveau widget des vérifications que nous croyions réservées au bout en bout, elle a rendu l’essentiel de la prévention des régressions abordable en quelques heures.

Bibliographie

Android Developers. (2026). *Test apps on Android*. Google. <https://developer.android.com/training/testing>

Atlassian. (2026). *Reference issues in your development work*. Atlassian Support. <https://support.atlassian.com/jira-software-cloud/docs/reference-issues-in-your-development-work/>

Conventional Commits. (2023). *Conventional Commits 1.0.0*. <https://www.conventionalcommits.org/en/v1.0.0/>

Firebase. (2026). *Distribute Android apps to testers using the Firebase console*. Google. <https://firebase.google.com/docs/app-distribution/android/distribute-console>

Flutter. (2026a). *Testing Flutter apps*. Google. <https://docs.flutter.dev/testing/overview>

Flutter. (2026b). *Integration testing*. Google. <https://docs.flutter.dev/testing/integration-tests>

GitHub. (2026). *Linking a pull request to an issue*. GitHub Docs. <https://docs.github.com/en/issues/tracking-your-work-with-issues/linking-a-pull-request-to-an-issue>

Humble, J. et Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

OWASP. (2024). *OWASP Mobile Application Security Verification Standard (MASVS)*. OWASP Foundation. <https://mas.owasp.org/MASVS/>

Schwaber, K. et Sutherland, J. (2020). *The Scrum Guide: The definitive guide to Scrum — The rules of the game*. <https://scrumguides.org/>

Vocke, H. (2018). *The practical test pyramid*. martinowler.com. <https://martinfowler.com/articles/practical-test-pyramid.html>

Avant de remettre : vérifiez chaque URL et ajustez les dates de consultation. Le seuil exigé est de 6 sources dont 4 techniques ou professionnelles ; la liste ci-dessus en compte 11, dont 9 techniques (Flutter ×2, Firebase, Android Developers, GitHub, Atlassian, Conventional Commits, OWASP, Scrum Guide).

Annexe A — Definition of Ready et Definition of Done
A.1 Definition of Ready (liste de vérification de conversation, non bloquante)
Une issue est <i>prête</i> à entrer au sprint si :
1. La valeur pour Eau Renaissance est formulée en une phrase (« pour que le client puisse... »). 2. Les critères d’acceptation sont écrits en Gherkin et comprennent au moins un cas d’erreur. 3. Les cas mobiles applicables sont tranchés : comportement hors ligne, permissions requises, état de chargement (CircularProgressIndicator

ou squelette), message d’erreur affiché à l’utilisateur.
4. Les dépendances sont nommées (point d’accès d’API disponible ? maquette validée ? données de test ?).
5. L’estimation est faite par l’équipe et l’issue tient dans un sprint ; sinon, elle est découpée.
6. Un membre autre que l’auteur peut reformuler l’issue sans poser de question.
Dérogation : l’équipe peut admettre une issue non prête par décision explicite consignée en commentaire, avec la raison et le risque accepté.
A.2 Definition of Done
Une issue est terminée seulement si tous les points suivants sont vrais :
- [] Les critères d’acceptation sont vérifiés un à un, y compris les cas d’erreur.
- [] Le code est sur une branche type/EAU-xx-sujet et fusionné par PR.
- [] La PR est approuvée par un pair selon la grille de l’annexe B.
- [] dart format, flutter analyze (aucun avertissement) et flutter test passent, localement et en intégration continue.
- [] Des tests unitaires couvrent la logique nouvelle et un test de widget couvre tout nouvel écran ; tout fix: est accompagné d’un test qui échouait avant.
- [] Aucune régression sur le parcours de recharge (integration_test ou grille manuelle de l’annexe C).
- [] Vérifié sur un appareil Android physique, depuis un APK release installable (jamais en mode debug).
- [] Aucune donnée sensible en clair : jeton dans flutter_secure_storage, rien de sensible dans les journaux.
- [] L’issue est liée à la PR et fermée automatiquement à la fusion ; le tableau est à jour.
- [] La fonctionnalité est démontrable en moins de 2 minutes par n’importe quel membre de l’équipe.

Annexe B — Grille de revue de PR

Règle de délai : toute PR reçoit une première réponse en moins de 24 h ouvrables. Une PR de plus de ~400 lignes modifiées doit être justifiée ou découpée.

#	Point de vérification	OK / À corriger
1	La PR est liée à une issue (Closes EAU-xx) et son titre porte la clé Jira	
2	Le périmètre correspond à l’issue : aucun changement hors sujet	
3	Les critères d’acceptation sont satisfaits (le relecteur les relit)	
4	Les messages de commit suivent <i>Conventional Commits</i>	
5	Tests présents et pertinents ; ils échoueraient sans	

	le code livré	
6	Cas d'erreur traités : réseau absent, réponse d'API en échec, permission refusée	
7	Aucune valeur secrète, clé ni URL de production dans le code	
8	Aucune donnée personnelle ni jeton dans les journaux	
9	Aucun <code>print</code> ni <code>debugPrint</code> oublié, aucun code mort, aucun <code>TODO</code> non tracé	
10	Nommage et structure cohérents ; <i>null safety</i> respectée, aucun <code>dynamic</code> ni <code>!</code> injustifié	
11	Rendu maîtrisé : <code>ListView.builder</code> pour les listes, constructeurs <code>const</code> , aucun <code>setState</code> reconstruisant tout l'écran, <code>dispose()</code> des contrôleurs	
12	Captures ou courte vidéo jointes si l'interface change	

Étiquettes de commentaire — `bloquant` (à corriger avant fusion), `suggestion` (améliorable, non bloquant), `question` (demande de clarification), `bravo` (à conserver). Ces étiquettes évitent qu'une remarque de style bloque une fusion.

Annexe C — Plan de tests minimal : fonctionnalité « Recharge de carte »

Périmètre. L'utilisateur authentifié associe une carte par code QR, recharge un montant, voit son solde mis à jour et retrouve l'opération dans son historique.

C.1 Tests unitaires (flutter_test , dossier test/unit/)

ID	Cas	Attendu
U1	<code>calculerNouveauSolde(2500, 2000)</code> — cents en <code>int</code>	4500
U2	Montant négatif ou nul	Erreur de validation, aucun appel réseau
U3	Montant au-delà du plafond <code>100 \$</code>	Rejet avec message explicite
U4	Montant saisi avec décimales (<code>45,50 \$</code>)	Converti en <code>int</code> de cents ; aucun <code>double</code> dans le calcul du solde
U5	Idempotence : même clé de transaction envoyée deux fois	Un seul crédit appliqué
U6	Formatage d'affichage (<code>4500</code> → <code>45,00 \$</code>)	Format monétaire québécois

C.2 Tests de widgets (flutter_test + mocktail / MockClient , dossier test/widget/)

ID	Cas	Attendu
I1	Saisie valide → appui sur « Confirmer »	<code>CircularProgressIndicator</code> affiché, bouton désactivé, écran de succès après <code>pump()</code>
I2	Le <code>MockClient</code> répond 500	<code>SnackBar</code> d'erreur lisible, solde inchangé, nouvelle tentative possible
I3	Réseau absent	Message hors ligne ; aucune écriture locale du solde comme si la recharge avait réussi
I4	Délai d'attente dépassé puis réponse tardive	Aucun double crédit affiché (réponse ignorée)
I5	Permission caméra refusée (<code>permission_handler</code> simulé)	Écran d'explication et saisie manuelle du numéro de carte proposée
I6	Historique après recharge	La nouvelle opération apparaît en tête, avec date et montant

C.3 Test de bout en bout (integration_test , dossier integration_test/) — un seul, le parcours de démo

E1 : connexion avec un compte de test → scan d'un code QR de carte (image injectée à `mobile_scanner` en mode test) → recharge de 20 \$ → le solde affiché augmente de 20 \$ → l'opération est visible dans l'historique. Lancement : `flutter test integration_test/recharge_test.dart -d <appareil>`. Exécuté avant chaque démo et sur chaque PR touchant la recharge. Repérage des éléments par `Key` explicites plutôt que par texte affiché, pour ne pas casser le test au moindre changement de libellé.

C.4 Grille exploratoire manuelle (10 min, appareil physique)

[[Appareil : modèle, version d'Android]]

1. Recharge en mode avion, puis réactivation du réseau → aucun double crédit.
2. Application mise en arrière-plan pendant la recharge, puis reprise → état cohérent.
3. Double appui rapide sur « Confirmer » → une seule opération.
4. Rotation de l'écran durant le chargement → aucun plantage, aucun état perdu (`AutomaticKeepAlive` / contrôleurs conservés).
5. Batterie faible / appareil bas de gamme → temps de réponse acceptable.
6. Fermeture forcée puis réouverture → solde rechargé depuis le serveur, non depuis un cache douteux.

Critères de sortie du plan. U1-U6 et I1-I6 au vert (`flutter test`) ; E1 au vert sur appareil physique ; grille manuelle sans anomalie bloquante. Hors périmètre : remboursements, reçus PDF, multi-cartes (au *backlog*).

Annexe D — Registre des blocages

ID	Blocage	Cause racine	Impact	Action prise	Palier d'escalade	Statut
B1	Build Android <i>release</i> en échec pendant 1,5 jour	Incompatibilité entre le plugin <code>mobile_scanner</code> , la version du SDK Flutter et le greffon Gradle ; <code>minSdkVersion</code> trop basse	Recharge non démontrable au sprint 2 ; <code>12</code> h perdues	Versions épinglées dans <code>pubspec.yaml</code> , <code>pubspec.lock</code> versionné, SDK Flutter figé par FVM ; APK de démo produit 48 h avant la démo	3 — publication dans le canal d'équipe	Résolu
B2	Solde désynchronisé devant le client	Aucune clé d'idempotence ; une nouvelle tentative après délai dépassé créditaient deux fois	Perte de crédibilité ; correctif en urgence	Clé d'idempotence par transaction côté API ; tests U5 et I4 ajoutés ; <code>fix</code> : accompagné d'un test	3	Résolu
B3	Scan de code QR instable sur [appareil bas de gamme]	Permission caméra demandée trop tard (<code>permission_handler</code> appelé après l'ouverture du scanner) et absence de secours ; contrôleur <code>mobile_scanner</code> non libéré au <code>dispose()</code>	Parcours inutilisable pour une partie des clients	Demande de permission avant l'ouverture du scanner, écran d'explication, saisie manuelle en repli (test I5), libération du contrôleur	2 — binôme	Résolu
B4	Demandes ajoutées pendant la démo (reçu PDF, remboursement partiel)	Aucun processus de capture de portée	Sprint 2 dépassé de <code>n</code> jours ; travail en cours inachevé	Saisie en direct comme issues <code>EAU-61</code> , <code>EAU-62</code> ; arbitrage au raffinement ; déroulement de démo fixe (R8)	3	Traité, au <i>backlog</i>
B5	[Blocage réel du sprint 3]	[]	[]	[]	[]	[]

Adaptez B1-B4 à vos blocages réels : l'évaluation porte sur l'authenticité du vécu, pas sur la gravité. Le minimum exigé est de trois blocages avec cause, impact et action.

Annexe E — Preuves d'application (gabarit à compléter)

Insérez ici les captures. Chaque preuve doit porter une légende indiquant **ce qu'elle démontre** et **à quelle recommandation** elle se rattache.

#	Preuve attendue	Où la prendre	Recommandation	Statut
P1	Tableau du sprint à jour (colonnes, éléments terminés)	Jira / GitHub Projects	R1-R2	[]
P2	Issue avec critères d'acceptation en Gherkin	Issue <code>EAU-53</code>	R3	[]
P3	Branche nommée selon la convention + historique de commits lisible	<code>git log --oneline --graph</code>	R1	[]
P4	PR liée à l'issue, avec commentaires de revue et grille appliquée	PR <code>#12</code>	R2	[]

P5	Commits significatifs (<code>feat:</code> , <code>fix:</code> avec son test)	[[liens de comparaison]]	R1-R4	[[
P6	Exécution des tests au vert (capture de la console ou de l'intégration continue)	<code>flutter test</code> / <code>flutter analyze</code> / Actions	R4-R9	[[
P7	Règle de protection de branche active	Paramètres du dépôt	R1	[[
P8	APK <i>release</i> installable et démo sur appareil physique	Console Firebase App Distribution + photo/vidéo	R8	[[
P9	Trace de journalisation avec identifiant de corrélation ou événement Sentry	Journaux serveur / tableau de bord <code>sentry_flutter</code>	R6	[[
P10	Demandes du client saisies en direct durant la démo	Issues [[EAU-61, EAU-62]]	R8	[[

Amélioration réellement implantée — à détailler (exigence C). Candidate recommandée : **la clé d’idempotence sur la recharge (B2)**, parce qu’elle se démontre en trois preuves : l’issue et son critère d’acceptation, le commit `fix:` avec le test U5 qui échouait avant, et le parcours rejoué sans double crédit. Documentez : situation avant → changement apporté → preuve → effet mesuré.

Annexe F — Plan de la présentation orale (8 à 10 min)

Temps	Contenu	Qui
0:00-1:00	Le client, l’application de carte rechargeable, et le problème vécu : solde désynchronisé en démo, commits sans traçabilité, portée mouvante	[[
1:00-1:45	Méthode en une diapositive : mesurer, comparer selon 4 critères, décider, appliquer, mesurer	[[
1:45-4:15	Décision 1 — Issue → branche → PR avec clé Jira. Ce qui a été comparé, pourquoi GitHub Flow, preuve à l’écran (PR liée à l’issue)	[[
4:15-6:15	Décision 2 — Tests ciblés sur le parcours de recharge. Pyramide adaptée à la taxonomie Flutter (unitaire / widget / intégration), arbitrage sur <code>integration_test</code> , preuve (test U5 + E1 au vert)	[[
6:15-8:00	Décision 3 — Déroulement de démo et capture de la portée. Build installable, saisie des demandes en direct, preuve (issues créées durant la démo)	[[
8:00-9:00	Impact mesuré (tableau avant/après), ce qui a coûté plus cher que prévu, ajustement prévu au sprint 4	[[
9:00-10:00	Contribution de chaque membre (commits, issues, livrables) + questions	tous

Conseils. Une seule diapositive par décision, avec la preuve visible à l’écran plutôt que du texte. Préparez le tableau avant/après comme diapositive de conclusion : c’est la partie qui distingue une recherche appliquée d’un simple résumé de bonnes pratiques. Chaque membre doit pouvoir nommer ses propres commits et issues.